

Doksli Lore InterupsiArOS

Untuk kalangan kami saja



Kelompok InterupsiArOS

Dipta	Raja Pertama
Bagas	Raja Kedua
Aqsha	Raja Ketiga
Sultan	Raja Terakhir

Sekolah Teknik Elektro dan Informatika - Komputasi
Institut Teknologi Bandung
2026/2027

Chapter 1

PENDAHULUAN

Dunia ini penuh dengan kegelapan, kebodohan, dan kepengecutan. Nilai-nilai inheren yang tumbuh di jiwa manusia sejak eksistensi mereka dihembuskan oleh Prometheus, jiwa-jiwa pejuang yang haus akan kemenangan dan tak tergoyahkan sekalipun oleh kehancuran tempat tinggalnya dan jatuhnya jasad rekan sejati mereka di medan perang, telah hilang. Kini, manusia berada di titik terendah spesies mereka. Sekumpulan organisme yang seharusnya merupakan puncak rantai makanan dari segala makhluk yang pernah hidup di muka bumi telah kehilangan status mereka sebagai entitas terkuat di bawah para dewa-dewi, menjadi makhluk terhina, yang eksistensinya merupakan pengkhianatan terhadap para dewa-dewi itu sendiri.

Itulah kata-kata yang keluar dari mulut Ares, Sang Dewa Perang yang Bengis dan Haus akan Kemenangan, namun Bijak dan Pendorong Keberanian. Kata-kata tersebut bukanlah sebuah kekecewaan semata, melainkan sebuah frustrasi yang diakumulasi oleh sang Dewa sejak kejatuhan masyarakat Yunani. Kata-kata tersebut adalah peringatan yang nyata bagi umat manusia di seluruh dunia.

Kami, 4 Raja Pejuang Agung, adalah titisan para SPARTANS yang senantiasa berjuang demi bara api perjuangan yang menuntut akal yang menakar, hati yang merasa, serta perjalanan yang panjang di depan mata para pejuang sejati. Kami adalah representasi dari Ares yang akan membawa semangatnya ke dunia fana ini. Kami membutuhkan sesuatu, agar nilai-nilai Ares dapat tersampaikan secara luas. Oleh karena itu, tertulis secara mentah pada surat ini, kami mengorbankan segala darah, air mata, dan keringat dalam penciptaan sebuah mahakarya kami yang dapat menciptakan tether kompatibilitas bagi Ares di dunia yang hina ini serta menyambungkan titah-titah Ares pada sinopsis seluruh umat manusia di muka Bumi ini baik yang masih hidup maupun sudah tiada. Melalui InterupsiArOS, kami akan menghidupkan kembali jiwa-jiwa semangat perang dan urat keberanian manusia dan membawa mereka kembali ke medan perang kami, para SPARTANS.

Tidak ada ruang untuk kelemahan dan ketidaksiapan di sini. Yang akan hidup hanyalah mereka yang dapat beradaptasi, memiliki gairah dan kekuatan untuk mengekspansikan kekuasaan bagai ekspansi kofaktor pada sebuah operasi invers matriks, serta mereka yang mempersiapkan diri untuk perjalanan yang tidak pasti.

Chapter 2

TOOLCHAIN, KERNEL, & GDT

2.1 How an OS starts up

Bagaimana booting sebuah OS komputer sebenarnya terjadi? Secara singkat, jawabannya adalah sebagai berikut:

1. Listrik mengalir ke hardware
2. CPU menjalankan firmware BIOS (Basic Input/Output System) yang terdapat di motherboard
3. BIOS menjalankan POST (Power-On Self Test) yang melakukan pengecekan terhadap fungsionalitas hardware
4. Jika POST berhasil, BIOS akan memuat MBR (Master Boot Record), yaitu sektor 512 bytes pertama pada bootable device (penyimpanan yang mengandung OS), ke RAM
5. MBR menjalankan kode Bootstrap, lalu Bootstrap memindahkan Bootloader dari bootable device ke RAM dan mengeksekusinya
6. Bootloader memindahkan OS ke RAM sehingga OS akhirnya dapat digunakan

Fungsinya bisa disimpulkan sendiri dari nama-namanya.

2.2 File-file awal aneh

File-file ini disediakan sedari awal.

2.2.1 grub1

File grub1 yang terletak di `other/grub1` adalah bootloader yang dipakai di Tubes OS ini.

2.2.2 kernel-entripoint.s

Kernel itu apa ya kak? Kernel itu adalah inti dari sebuah OS yang menjalankan hal-hal seperti syscall, manajemen memori, scheduling, dll. Basically kernel adalah core dari penyambung interface dengan hardware.

Perlu diketahui bahwa kernel tidak bisa langsung membaca bahasa C, namun harus melalui kode Assembly terlebih dahulu, yang dalam kasus ini adalah kernel-entripoint.s itu sendiri (dibantu loader juga, tapi itu nanti).

Secara umum, yang dilakukan oleh kernel-entripoint.s adalah set-up agar loader dan GRUB bisa melakukan pekerjaannya. kernel-entripoint.s menyediakan Call Stack juga agar bahasa C dapat digunakan. kernel-entripoint.s juga melakukan load_gdt.

2.2.3 kernel-entripoint.h

Not to be confused with kernel-entripoint.s, tapi ini header file buat eksekusi load_gdt.

2.2.4 gdt.h

Header untuk... GDT. Nanti dibahas lagi.

2.2.5 string.h & string.c

Ini udah kepahe belum yah

2.2.6 Makefile

Ya basically buat automate compiling and linking and all that aja sih.

2.3 Kernel and Linker

2.3.1 kernel.c

Basically main programnya. Atur-aturlah.

2.3.2 linker.ld

Basically yang melakukan menggabungkan objek hasil kompilasi jadi satu binary executable. Entry dilakukan dari loader. Kode di-run dengan batas bawah 0x00100000 (1 MB) serta aligning by 4 KB lots of things like:

- `.multiboot`
Atur-atur lah, multiboot header supaya kernel bisa dicari sama GRUB
- `.text`
Menyimpan kode program
- `.rodata`
Menyimpan read only
- `.data`
Menyimpan global variabel
- `.bss`
Menyimpan uninitialized variables

2.4 Buat apa Makefile?

Seperti yang dijelaskan sebelumnya, fungsi Makefile adalah untuk melakukan kompilasi dan linking serta as a whole mengotomasi proses building proyek.

```
ASM                = nasm

UNAME_S            := $(shell uname -s)
ifeq ($(UNAME_S), Darwin)
LLVM_PREFIX        := $(shell brew --prefix llvm)
CC                 = clang
LIN                = $(shell brew --prefix lld)/bin/ld.lld
II                 = mkisofs
ARCH_CFLAG         = --target=i386-elf -march=i386 -mno-mmx -mno-
                    sse -mno-sse2 -mno-avx -msoft-float -Wno-unused-command-
                    line-argument
else
CC                 = gcc
LIN                = ld
II                 = genisoimage
ARCH_CFLAG         =
endif
```

Basically ini buat setup compiler, linker, dan tool genisoimage. Dibedakan untuk macOS dan Linux yang bisa dilihat dari ifeq dan else-nya.

```
SOURCE_FOLDER = src
OUTPUT_FOLDER = bin
```

```
ISO_NAME      = OS2025
```

Ini buat lokasi folder source, output, dan file name iso yang ingin dibuat

```
WARNING_CFLAG = -Wall -Wextra -Werror
DEBUG_CFLAG   = -fshort-wchar -g
STRIP_CFLAG   = -nostdlib -fno-stack-protector -nostartfiles
               -nodefaultlibs -ffreestanding
CFLAGS        = $(DEBUG_CFLAG) $(WARNING_CFLAG) $(STRIP_CFLAG)
               $(ARCH_CFLAG) -fno-builtin -m32 -c -I$(SOURCE_FOLDER)
AFLAGS        = -f elf32 -g -F dwarf
LFLAGS        = -T $(SOURCE_FOLDER)/linker.ld -melf_i386
IFLAGS        = -R -b boot/grub/grub1 -no-emul-boot -boot-
               load-size 4 -A os -input-charset utf8 -quiet -boot-info-
               table
```

CFLAGS buat GCC, AFLAGS buat ASM, LFLAGS buat Linker, IFLAGS buat ISO maker. Probably ga perlu terlalu paham terlalu banyak terkait flags-flagsnya.

```
run: all
      @qemu-system-i386 -s -S -cdrom $(OUTPUT_FOLDER)/$(
      ISO_NAME).iso
all: build
build: iso
clean:
      rm -rf $(OUTPUT_FOLDER)/*.o $(OUTPUT_FOLDER)/*.iso $(
      OUTPUT_FOLDER)/kernel
```

run untuk **build** dan **run** OS dari ISO pakai **qemu**. **all** and **build** harusnya melakukan hal yang sama, yaitu membuat disk image ISO, also does **kernel**. **kernel** buat ngurus ASM, GCC, dan Linker. **clean** buat bersihin file output.

PASTIKAN SETIAP C FILE BARU TAMBAHIN DI COMPILE KERNEL!!!

2.5 Global Descriptor Table (GDT)

Basically, GDT adalah tabel yang berisi Segment Descriptor. Segment Descriptor ini adalah tipe data yang menyimpan informasi-informasi tentang suatu segmen, which is daerah memori linier. Informasi yang disimpan terutama adalah flag-flag dan tingkat hak akses tertentu. Di proyek ini, kita pakai Descriptor Privilege Level (DPL) Ring 0 untuk Kernel Mode dan Ring untuk User Mode.

2.5.1 gdt.h dan gdt.c (gabung aja lah ya)

- `GDT_MAX_ENTRY_COUNT` 32
Ukuran maksimal Segment Descriptor dalam suatu GDT
- `GDT_KERNEL_CODE_SEGMENT_SELECTOR`
Indeks Segment Tipe Code (nantu dijelaskan), whichis Indeks Pertama whichis `sizeof(SegmentDescriptor)` whichis 8 bytes. 8 bytes kali 1 = 0x8
- `GDT_KERNEL_DATA_SEGMENT_SELECTOR`
Indeks Segment Tipe Data (nantu dijelaskan) whichis indeks kedua jadi 0x10 aka 16.
- `SegmentDescriptor`
Struct representasi SegmentDescriptor. SegmentDescriptor berukuran 8 byte. Bisa dilihat memanfaatkan kemampuan C untuk menyesuaikan ukuran bit suatu tipe data (sintaks bit-field). Isi SegmentDescriptor dideskripsikan in a bit. Also notice, di struct ini dan beberapa struct lainnya ada `__attribute__((packed))`. Fungsinya adalah supaya ga ditambahin padding of any sorts jadi ukurannya memang udah fix segitu.
- `GlobalDescriptorTable`
Memuat array berukuran entri maksimum tipe data SegmentDescriptor
- `GDTR`
Basically register yang memuat informasi terkait GDT supaya bisa dicari CPU di RAM. Memuat ukuran GDT dan alamatnya. Note ada extern struct GDTR bernama `__gdt_gdtr`, basically dibuat variabel global shared lah. Sizenya `sizeof gdt` dikurangi 1

Gambarnya liat sendiri di Buku dan Intel Manual.

Jika kita bagi dari segi definisi dan fungsinya, ada bagian-bagian berikut:

- Segment Limit Field
Ukuran dari segment. Diwakili oleh `segment_low` di bit 0-15 di 16 byte pertama dan `segment_high` di bit ... itung sendiri udah gede. Ukuran juga itung sendiri be.
- Base Address Field
Menyatakan alamat fisik awal dari lokasi segmen. Diwakili oleh `base_low`, `base_mid`, dan `base_high`.

- Type Field
Tipe dan hak akses segmen. Misalnya executable (code), readable, atau writable. Diwakili oleh `type_bit`.
- S (Descriptor Type) Flag
Jika 1, segmennya kode/data. Jika 0, segmennya sistem. Karena di kita kode/data semua, jadi dinamain `non_system`.
- DPL Flag
Ring-ring DPL-nya. Diwakili oleh privilege.
- P (Present) Flag
Jika segmen ada di RAM, nilainya 1. Diwakili oleh `valid_bit`.
- AVL (Available and Reserved Bit) Flag
Available bit, sesuaikan saja SPARTANS. Diwakili `avl`.
- L (64-Bit Code) Flag
Digunakan pada arsitektur x64. Di kita, nilainya 0 karena kita bikin sistem x86. Diwakili `long_mode`.
- D/B (Default Operation Size/Default Stack Pointer And/Or Upper Bound) Flag
Basically ngatur ukuran operasi (executable segment), ukuran stack pointer, dan upper bound segment. Nilainya 1 jika 32-bit system, 0 jika 16-bit. Diwakili oleh `opr_32_bit`.
- G (Granularity) Flag
Scaling segment limit. Kalo 0, pake unit per byte jadi maks 1 MB. Kalo 1, per 4KB jadi maks 4GB. Diwakili oleh granularity.

Terlihat susunannya seperti di bawah.

```
// First 32-bit
uint16_t segment_low;
uint16_t base_low;

// Next 16-bit (Bit 32 to 47)
uint8_t base_mid;
uint8_t type_bit    : 4;
uint8_t non_system  : 1;
uint8_t privilege   : 2;
uint8_t valid_bit   : 1;

uint8_t segment_high : 4;
```

```
uint8_t avl          : 1;
uint8_t long_mode    : 1;
uint8_t opr_32_bit   : 1;
uint8_t granularity  : 1;
uint8_t base_high;
```

2.5.2 Load GDT

Di kernel.c, kita cukup menambahkan:

```
void kernel_setup(void) {
    // ...
    load_gdt(&_gdt_gdtr);
    // ...
}
```

2.5.3 Bonusan aja sih

Di gdt.c dan possibly beberapa file ke depan, kita pakai yang namanya Designated Initializers (pokoknya depannya pakai titik). Teknik ini dipakai supaya langsung mengacu ke variabel di struct pada posisi address yang bersesuaian terlepas dari urutan kita menginputnya.

Sesuai tabel GDT yang kita buat, indeks ke-0 diisi GDT null, indeks ke-1 buat segmen Kernel Code, indeks ke-2 buat segmen Kernel Data.

Aturan-aturannya bisa dilihat sendiri di buku.

Kernel Code bersifat Read-only and Executable dan berfungsi menyimpan instruksi binary yang dikompilasi. Kernel Data bersifat Read-write and Non-executable dan berfungsi menyimpan variabel global, stack kernel, dan system-wide structures.

Ya buzzword buzzword dikit. Tapi FYI aja.

2.6 Awal Mula Kebangkitan Ares

Dunia akan digetarkan oleh prosedur-prosedur yang kami buat demi cinta kami kepada kota kami terdahulu, SPARTA. Persiapan kami dalam membangkitkan kembali mimpi-mimpi mulianya akan memberikan esensi derap langkah pejuang-pejuang yang berjiwa ksatria dan pantang menyerah terus berjuang bersama bersatu berhimpun dalam menghadapi musuh-musuh pengecut yang akan terpecut oleh keberanian dan keinginan untuk memperluas daerah kebengisannya menuju pusat ekonomi yang maju dari robohan kehancuran peradaban yang lemah. Awal mula ini menjadi momentum bagi kita semua untuk mengapresiasi bagaimana persiapan kami dalam meneteskan pengaruh

sang Dewa Perang di kehidupan modern ini merupakan fondasi dalam pembentukan dunia yang terstruktur, berdaulat, adil, dan makmur.